

Generation, retrieval-augmented systems, and where this leaves you

Dataset: COCO and the Part V corpora

How to use these notes

The primary text for this lecture is Géron, Chapters 15–16. These notes are not a replacement for it and do not repeat it: they are the lecture’s own argument written out at length — the derivation done slowly, the numbers with the runs that produced them, and the reasoning between one slide and the next that a deck can only gesture at. Read the chapter for the method; read these for why the lecture makes the choices it does. Every figure quoted here is printed by this lecture’s notebook, and the course checks that mechanically rather than trusting it.

Contents

1	Two weak spots, both about entries	2
2	Filling the holes	2
3	Retrieval-augmented generation	3
3.1	Decide how you will check it first	4
4	The course, as one argument	5
4.1	The eighteen derivations, and where each returned	6
4.2	The failures, collected	6
4.3	What this course did not cover	7
5	What you can do now	8
6	The notebook	8
7	Exercises	9

1 Two weak spots, both about entries

Lecture 23 built a catalogue you can search with a sentence, and left two things unfixed. Some entries have *no usable description*, so there is nothing for a text tower to encode — and a text index cannot rank a document that does not exist, so those entries score exactly 0%, structurally rather than statistically. And some queries are *ambiguous*, where a ranking is a poor answer to a question the user has not finished asking.

Both are repaired by generation. A multimodal model can write the missing description; and a generator with the retriever attached can answer the ambiguous query in words, grounded in what the catalogue actually contains.

2 Filling the holes

60 of the 200 entries have no description. The captioner is architecturally the encoder–decoder of Chapter 15 with an image encoder in place of the text one — a vision transformer, a transformer decoder cross-attending to it, trained on image–caption pairs by next-token prediction, 224 million parameters. Nothing new; a different pair of modalities in the same box. Nine lines, and 21 seconds for 60 images with beam search — not instant, so run it offline, once per new entry.

Human description (deleted)	Generated description
This wire metal rack holds several pairs of shoes and sandals	a dog laying on top of a pair of shoes
A green vase filled with red roses sitting on top of table	a vase of flowers sitting on a window sie
A giraffe bending over while standing on green grass	the gi is brown and white

The first row is the interesting one: *both sentences are true*. There is a dog asleep on the shoe rack, and the two writers chose different subjects. The third row is not a sentence at all.

Text-only route, all 200 queries	R@1	R@10
Every entry described by a person	61.5%	96.0%
60 entries with nothing	46.0%	68.5%
Those 60 auto-captioned	61.5%	94.0%

At rank 1 the recovery is exact — the auto-captioned index returns the same 61.5% as the one every person wrote, which is a tie rather than a win. At rank 10 it is *not* complete, and that gap is the real finding.

Meanwhile, the route that never read a description

On the same 60 entries, the text index with descriptions deleted scores 0%, the auto-captioned index recovers most of it, and the joint-space image route — which never needed text at all — scores 80.0%.

Two routes that fail for different reasons are worth more than one better route. The image route is immune to missing text; the text route is immune to a photograph that does not show the distinguishing feature.

Failure condition — three things not to claim

A generated caption is *not evidence about the product*. It is a restatement of the photograph, and it will confidently name things that are not there.

The recovery is measured on 60 entries of 200 — a small sample with a wide interval. And we are scoring generated captions against human captions of the same image, which is a friendly test: a customer's wording is not a COCO caption.

The failure mode to watch for: an auto-caption that is *wrong* makes an entry findable under the wrong query, which is worse than unfindable. Nothing we measured detects that.

So the honest use is narrow and worth stating: fill holes, mark the filled entries as machine-written, and measure retrieval with and without them.

Prompting is a hyperparameter

A generator's output depends on how you ask, and the difference is often larger than the difference between models. Trying five prompts and keeping the one with the best test score is choosing a hyperparameter on the test set — Lecture 2's error, in a costume that does not look like modelling at all.

Report the prompt with the result. A number without the prompt that produced it is not reproducible, and prompts are rarely written down because they do not feel like code.

3 Retrieval-augmented generation

The user asks in words; the retriever of Lecture 20 embeds the question and returns the k most similar entries; those go into the generator's context with the question; and the generator answers *from what it was given*, citing which entry it used.

Why this is not just a bigger model: *the knowledge lives in the index, not in the weights*. Add a product and it is answerable immediately; remove one and it stops being cited. No retraining — and the system can say where an answer came from, which is usually the requirement that decides whether it can be deployed at all.

	Good at	Bad at
Retriever	finding the right entries, fast, over millions	saying anything; it only ranks
Generator	fluent answers, synthesis across entries	knowing anything reliably; it will produce a plausible answer regardless

Failure condition — visible from neither half

Retrieved but ignored — the right entry is in the context and the generator answers from its own weights instead.

Not retrieved, answered anyway — the retriever misses and the generator does not say so, producing a fluent answer with nothing behind it.

Retrieved and misread — the entry is present and correct, and the answer misstates it.

Evaluating the retriever alone by $\text{recall}@k$ and the generator alone by fluency can both look excellent while the system does all three. *The join needs its own measurement.*

Two upstream decisions are usually made without measurement. *Chunking*: too small and a chunk lacks the context that makes it answerable; too large and the embedding averages several topics and matches none well; split badly and a sentence cut in half retrieves for neither of its halves. It is a hyperparameter — vary it, hold everything else fixed, and measure $\text{recall}@k$ on the same judgements. And *how many to retrieve*: small k risks the right entry not being there at all, large k buries it among distractors — which is where re-ranking earns its cost, retrieving generously with the cheap bi-encoder and re-ranking with the cross-encoder, high recall from the first stage and high precision from the second.

3.1 Decide how you will check it first

“The answer is good” is not measurable in a lecture. So give every entry a stock number and require the assistant to cite them: a cited SKU either exists in the catalogue or it does not. Count the citations, count the ones that exist, report the fraction — no judgement call anywhere in the loop.

This measures grounding, not helpfulness. Say which one you measured; they are not the same thing and only one of them is cheap.

Over 12 ambiguous queries	Closed book	Retrieval-augmented
Queries that cited at least one SKU	10	12
Citations emitted	32	24
Citations that exist	0	24
Grounding rate	0%	100%

Closed book, the model produces a fluent recommendation and cites stock numbers *in exactly the right format*. Across twelve queries it emitted 32 citations, of which none corresponds to an entry that exists.

The failure is not that it refuses

It is that it does *not* refuse, and nothing in the output distinguishes an invented SKU from a real one.

The generator is a 494-million-parameter instruction model, chosen so the whole thing runs on a laptop; a larger one would follow the citation instruction more reliably and would not change the shape of the result.

Failure condition — what this does not fix

Grounding is not correctness — a cited entry can exist and still be a bad recommendation. We measured the cheap half.

The retriever is now the ceiling: if the right entry is not in the top five, no amount of generation recovers it, which is why the R@5 measured last lecture is the number that matters here.

And fluency is unchanged. A wrong answer built from real SKUs reads better than a wrong answer built from invented ones.

Four ways to measure the join: *groundedness*, is every claim supported by a retrieved entry; *attribution*, does the cited entry actually say it — a citation that does not support its claim is worse than none; *abstention*, when nothing relevant is retrieved does it decline, measured deliberately with queries whose answer is genuinely absent; and only then, end to end, is the answer right. Plus the closed-book control: run the same questions with retrieval off, and whatever the system still answers correctly it knew already. *The difference is what retrieval actually bought* — the same shape as every ablation in this course.

Route	Before	After
Text query to image, joint space	73.0%	73.0% (untouched)
Text index, on the entries with no description	0%	recovered
Ambiguous queries, grounding rate	0%	100%

Two repairs, two measured deltas, and *one number deliberately unchanged as a control*. That last row is what makes the other two believable.

4 The course, as one argument

What the first lecture promised

“How to build a machine learning system, and how to know whether it works. Fitting a model is a few lines. Establishing that the number it reports means what it appears to mean is the whole of the rest of the course.”

That was a claim, not a plan. The claim, stated precisely: *every method you met was introduced at the moment a number you produced was wrong without it*. Not “was useful”. Not “is standard”. Was wrong, and measurably so, and you were the one holding the wrong number.

4.1 The eighteen derivations, and where each returned

Lecture	The derivation	Where it came back
2	least squares, the normal equation	ridge in 5; collinearity everywhere
3	imbalance; precision is not monotone	the maximum inside AP, Lecture 14
4	gradient descent	every network from Lecture 9 on
5	bias–variance	every learning curve since
6, 7	impurity; variance of a correlated average	multi-head attention, Lecture 18
8	PCA via the SVD; Johnson–Lindenstrauss	factorisation in 21; the sphere in 23
9	what a layer computes	everything after it
10	reverse-mode autodiff	why training is affordable at all
11	variance propagation	BPTT in 16; residuals in 18
12	weight sharing, equivariance, memory	what a ViT gives up, Lecture 23
14	IoU; mAP	ranking metrics in 19
15	stationarity; autocorrelation	causal masking in 18
17	softmax, cross-entropy, logits	attention’s softmax in 18; the contrastive loss in 23
18	scaled dot-product attention	Part V’s encoders, and 23
19	MRR, AP, NDCG	a ranking needs its own metrics
21	matrix factorisation and the SVD	Lecture 8, with missing entries
23	the contrastive objective and its temperature	the same two towers, different modalities

Lectures 20, 22 and 23 are three uses of *one object* — two encoders and an inner product. That is the strongest single thread in the course, and it is why Part V sits after transformers rather than before.

4.2 The failures, collected

Failure	First met	Measured cost
Fitting before splitting	Lecture 2	nothing measurable — and unknowable in advance
Evaluating on training data	Lecture 2	an RMSE of zero
Accuracy under imbalance	Lecture 3	a useless model above 90%
Choosing on the test set	Lectures 2, 5	an optimistic report, every time
Missing <code>zero_grad()</code>	Lecture 10	76.7 points
Default initialisation, deep	Lecture 11	a network that does not train at all
Random split on a series	Lecture 15	optimistic, and undeployable
Augmenting the validation set	Lecture 13	1.57 points of noise on the criterion
Embeddings compared without normalising	Lecture 23	a different ranking

Not one of them raises an exception. That is the whole reason the course is organised around measurement.

Two of them are worth restating with their arithmetic. In detection, the same detections at the same IoU threshold give 0.659 when averaged over classes and 0.817 when averaged over images — *a mean of a mean is not a mean*, and the headline moves by 0.157 depending on which you pick. Against the stricter 0.5-to-0.95 average of 0.439 the apparent gap looks like 0.378, and that comparison is not a comparison at all, because the two rows are averaged over different thresholds.

And in forecasting: the same model on the same rows scores 44,925 under a random split and 56,446 under a time-ordered one — 25.6% optimism, with nothing fitted on the test set and no preprocessing crossing the split. The leak was *the ordering*, and with positive autocorrelation a point's own near-future was in its training set.

What did not change, in twenty-four lectures

1. Split before anything is fitted.
2. Compute the trivial baseline first — a metric with nothing beside it is decoration.
3. Two numbers measured on different rows are not comparable, however similar the column headings look.
4. A difference smaller than the spread is not a difference. Report the spread.
5. Change one thing, or the difference is not attributable to anything.

Five rules, no mathematics, and they would have caught every failure in the table above.

4.3 What this course did not cover

“I took a machine learning course” is read by other people as a claim about what you can do, so you should be able to say precisely what it does and does not mean. The scope was Chapters 1–16 of one book, thoroughly, on thirteen datasets. That is a large and useful slice of applied machine learning. It is not the field.

Missing, and each is a course: reinforcement learning, a different problem shape entirely; generative image models; causality — every method here predicts, and none tells you what happens if you *intervene*; fairness and model governance; and serving and drift.

The two you are most likely to need first

Drift. Every number in this course was measured on data drawn like the training data. In production that stops being true, gradually and without warning, and the score you reported at launch is the last honest one anybody has. Monitor the inputs, not just the outputs — the distribution moves before the accuracy does.

Who it is worse for. Lecture 2 sliced the error by district and found the poorest tenth predicted worst. That slice was one line of code, and it is the difference between a system you can defend and one you cannot. Always ask for whom the average is wrong.

And the part that moves fastest: pretraining at scale — we used four pretrained models in this last application and trained none of them; instruction tuning, which is why the generator behaves as it does; and evaluating generated text, where we measured grounding *because grounding is checkable* and measuring whether an answer is good is unsolved.

The transferable part

Every one of those still fails in the ways this course taught you to look for: the wrong metric, a missing baseline, a test set that leaked, and a number quoted from one run.

Without a ground truth you can still measure *consistency* — ask the same thing twice in two wordings, and disagreement bounds reliability from above; *groundedness* against the retrieved text, mechanically; *abstention* on deliberately unanswerable queries, the cheapest useful test of a RAG system; and human judgement on a sample, expensive and still the anchor everything else is validated against.

5 What you can do now

Take a problem, frame it, and say what a correct answer would be before fitting anything. Choose a metric that matches the brief, and measure the trivial baseline first. Build the simplest thing that runs, inside a pipeline, evaluated on a protocol fixed in advance. Read the failure — slice the error, look at the worst cases, name the conditions under which the method breaks. Improve one thing at a time and report the ablation rather than the best row. And write down what the number was measured on, every time.

Six steps. Nothing in them is specific to a library, and none will be out of date in ten years.

A last word on the assistant

You will build most of this with an assistant, and Lecture 1 set out how: specify, generate, read, test, verify — and steps three to five are yours.

What has changed by now is that you know what to look for. “What was this measured on?”, “what is the baseline?”, “was anything fitted before the split?” are the questions an assistant will not ask itself, and they are precisely the ones that decide whether the number it hands you means anything.

The code was never the scarce part. Knowing whether to believe the output is, and that is what these twenty-four lectures were for.

How to keep learning: reproduce something — take a paper’s headline number and try to get it, and you will learn more from the gap than from the paper. Keep the habit of the baseline. Read the evaluation section first: if it does not say what was held out and how, the rest is decoration. And write the check before the model.

The libraries in these notebooks will be superseded. The derivations will not, and neither will the habit of asking what a number was measured on.

6 The notebook

What to change

Ask a question whose answer is not in the catalogue. A system that answers it anyway has the second failure of the join — and you have just built the test that finds it.

7 Exercises

1. An entry with no description scores 0% on a text retrieval route. State why this is a structural rather than a ranking failure, and give two independent repairs. [4]
2. Name the three failures that belong to the join of a retriever and a generator, and explain why evaluating each half separately cannot find them. [5]
3. A retrieval-augmented system is judged by a grounding rate. State precisely what that measures, what it does not, and the control you would run beside it. [5]
4. The same detections give mAP 0.659 averaged over classes and 0.817 averaged over images. Explain the discrepancy, and say which number you would report and why. [4]
5. Give the five procedural rules this course did not change, and for each name one failure it would have caught. [5]

Summary

Generation repaired the two holes Lecture 23 left. Captioning took entries that were structurally invisible to a text index — not badly ranked, absent — and made them findable, recovering the rank-1 figure to within one query in 200 of what human writers achieved and *not* recovering rank 10, which is the finding rather than the headline. Meanwhile the image route, which never needed a description at all, scored 80.0% on those same entries: two routes that fail for different reasons are worth more than one better route.

Retrieval-augmented generation took twelve ambiguous queries from a 0% grounding rate to 100%. Closed book, the model emitted 32 citations in exactly the right format, of which none existed — and the failure is not that it refuses but that it does not, with nothing in the output distinguishing an invented identifier from a real one. Grounding is not correctness; we measured the cheap half, and said so.

And a control row deliberately left unchanged is what makes the two measured deltas believable.

The course was one argument: every method arrived at the moment a number was wrong without it. Eighteen derivations, thirteen datasets, and a catalogue of failures none of which raises an exception — a scaler fitted early that cost nothing measurable, a missing `zero_grad()` that cost 76.7 points, a random split on a series that cost 25.6% and deployability. The five procedural rules that would have caught all of them contain no mathematics at all.